MODULE 6 • TRANSACTION CONTROL LANGUAGE (TCL)

COMMIT, ROLLBACK & SAVEPOINT

Comprehensive trainer study notes, real-world execution models, and 20 hands-on practice challenges for database transactions.

Instructor: **Sai Kumar Peddamodula** • SQL Masterclass

1. What is Transaction Control Language (TCL)?

In SQL, commands are classified into categories like DDL (structure), DML (data manipulation), and DQL (querying). **TCL (Transaction Control Language)** commands manage the changes made by DML statements (`INSERT`, `UPDATE`, `DELETE`).

> ⚠ **The AutoCommit Reality:** By default in MySQL and PostgreSQL, `autocommit = 1`. Every single line of SQL you run is committed to disk immediately! To gain undo protection, you must use explicit transactions.

Command	Full Meaning	Purpose in SQL	Can it be Undone?
`COMMIT`	Make Changes Permanent	Saves all staged changes to physical disk and releases locks.	NO (IRREVERSIBLE)
`ROLLBACK`	Cancel & Discard	Erases all staged changes since `START TRANSACTION`.	RESTORES STATE
`SAVEPOINT`	Create Checkpoint	Sets a named bookmark to allow partial rollback inside a transaction.	FLEXIBLE MARKER

💾 2. The COMMIT Command: Sealing the Deal

When you run `COMMIT`, the database engine flushes your staged changes from the active buffer (RAM) to the durable **Redo Log** on disk. All locks held on rows or tables are released.

```
-- Real-World Bank Transfer Scenario START TRANSACTION; -- Step 1: Deduct from Alice
UPDATE accounts SET balance = balance - 500 WHERE account_id = 'ACC_ALICE'; -- Step
2: Add to Bob UPDATE accounts SET balance = balance + 500 WHERE account_id =
'ACC_BOB'; -- Step 3: Record audit entry INSERT INTO transfer_log (from_acc, to_acc,
amount, transferred_at) VALUES ('ACC_ALICE', 'ACC_BOB', 500, NOW()); -- Finalize
permanently: COMMIT;
```

✅ **Trainer Rule:** Once `COMMIT` succeeds, the transaction is closed. Running `ROLLBACK` immediately afterward will do nothing because there are no open transactions to undo.

🔄 3. The ROLLBACK Command: The Emergency Undo Brake

`ROLLBACK` reads the database engine's **Undo Log** and applies inverse operations to undo every statement executed since the start of the transaction.

```
-- Accidental Mass Update Prevention START TRANSACTION; -- Dangerous Query: Missing
WHERE Clause! UPDATE employees SET salary = 0; -- Check affected count: 50,000 rows
changed! Panic! -- No problem — we are inside a transaction: ROLLBACK; -- Verify:
Every single employee's salary is 100% restored! SELECT COUNT(*) FROM employees WHERE
salary > 0;
```

🛑 **Production Gotcha:** Always pair your application code (Python, Java, Node.js) with `ROLLBACK` inside your `catch / except` blocks. If an error occurs and you forget to rollback, open locks can freeze your entire database!

🚩 4. SAVEPOINT & Partial Rollbacks

In complex enterprise workflows (e.g. flight booking, e-commerce checkout), you may have 10 sequential operations. If step 9 fails, rolling back all 10 steps frustrates customers. **SAVEPOINT** lets you roll back only to a specific bookmark without losing prior successful work.

```
START TRANSACTION; -- Step 1: Book Flight Seat UPDATE flights SET seats_left =
seats_left - 1 WHERE flight_no = 'AI-101'; -- Checkpoint 1: Seat is confirmed
SAVEPOINT seat_booked; -- Step 2: Attempt Special Meal Reservation UPDATE catering
SET vegan_meals = vegan_meals - 1 WHERE flight_no = 'AI-101'; -- Error: Out of vegan
meals! -- Rollback ONLY the meal — KEEP the flight seat! ROLLBACK TO SAVEPOINT
seat_booked; -- Complete the flight booking without meal: COMMIT;
```

Syntax	Behavior	State of Transaction
`SAVEPOINT name;`	Creates a named bookmark inside current transaction.	Transaction remains **ACTIVE**
`ROLLBACK TO SAVEPOINT name;`	Reverts statements executed after bookmark; keeps statements before it.	Transaction remains **ACTIVE**
`RELEASE SAVEPOINT name;`	Removes bookmark from memory (does not commit or rollback).	Transaction remains **ACTIVE**

⚠ 5. Implicit Commits (The Hidden Trap in SQL)

Certain SQL commands cause the database engine to automatically run an **Implicit Commit** behind your back. If you run any of these inside an active transaction, your prior work is immediately committed and cannot be rolled back!

Command Category	Examples That Cause Implicit Commit	Can You ROLLBACK?
DDL (Data Definition)	`CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`, `TRUNCATE TABLE`	NO — INSTANT COMMIT!
Transaction Initiation	Running `START TRANSACTION` while another transaction is open	AUTO-COMMITS PREVIOUS ONE!
Security & User Rights	`GRANT`, `REVOKE`, `CREATE USER`	INSTANT COMMIT!

🎯 6. 20 Hands-On Practice Scenarios (Self-Assessment)

Put your transaction knowledge to the test. Write clean, production-ready SQL scripts for each scenario below using `START TRANSACTION`, `COMMIT`, `ROLLBACK`, and `SAVEPOINT`.

1. Safe Balance Deduction

LEVEL: BEGINNER

Deduct $250 from customer #101. Write a transaction that checks if balance falls below 0, and if so, rolls back all changes.

> **Requirements:** Use `START TRANSACTION`, `UPDATE`, check logic, and `ROLLBACK`.

2. Order Placement & Inventory Sync

LEVEL: BEGINNER

Insert a new order into `orders` and decrease stock in `products` by 2 units. Ensure both succeed or neither does.

> **Requirements:** Atomicity guarantee via `COMMIT`.

3. Accidental Bulk Deletion Safeguard

LEVEL: BEGINNER

Simulate a scenario where an admin accidentally runs `DELETE FROM audit_logs;` without a `WHERE` clause inside a transaction, realizes the mistake, and recovers all records.

> **Requirements:** Demonstrate full recovery using `ROLLBACK`.

4. User Registration with Profile Creation

LEVEL: BEGINNER

Create a transaction that adds a user to `users` table, then creates a linked row in `user_profiles`. Commit only if both rows insert without foreign key errors.

> **Requirements:** Two related `INSERT` statements with `COMMIT`.

5. Single Savepoint Cart Modification

LEVEL: INTERMEDIATE

Add 2 items to an online cart. Set a savepoint. Add a 3rd item that turns out to be out of stock. Rollback to the savepoint and commit the first 2 items.

> **Requirements:** `SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, and `COMMIT`.

6. Multi-Savepoint Hotel Booking

LEVEL: INTERMEDIATE

Book a hotel room (Savepoint A). Add airport pickup (Savepoint B). Add luxury spa treatment. If spa treatment fails, cancel spa but keep airport pickup and room.

> **Requirements:** Multiple savepoints in a single open transaction.

7. Discount Coupon Rollback

LEVEL: INTERMEDIATE

Calculate order total ($120). Set a savepoint. Apply a promo code for 50% off. If coupon code validation rules fail, rollback discount but keep original order amount.

> **Requirements:** Partial rollback without aborting the checkout flow.

8. Subscription Renewal with Payment Log

LEVEL: INTERMEDIATE

Renew user membership by adding 30 days to `expiry_date`. Insert transaction record into `payment_history`. If payment record fails, revert the renewal.

> **Requirements:** Date arithmetic inside transaction with rollback safety.

9. Warehouse Stock Relocation

LEVEL: INTERMEDIATE

Transfer 100 laptops from Warehouse North to Warehouse South. Deduct from North, add to South. Verify total inventory across both warehouses remains unchanged.

> **Requirements:** Balanced two-sided stock transfer under transaction.

10. Release Savepoint Memory Cleanup

LEVEL: INTERMEDIATE

Create 3 successive savepoints during an import of 5,000 records. Once stage 2 is confirmed valid, release savepoint 1 to free database engine memory.

> **Requirements:** Proper usage of `RELEASE SAVEPOINT`.

11. Loan Repayment with Penalty Calculation

LEVEL: ADVANCED

Apply late penalty fee to loan account. Set savepoint. Apply customer payment. If payment amount is insufficient to cover penalty plus interest, roll back payment but retain penalty fee.

> **Requirements:** Conditional savepoint rollback based on business logic.

12. Preventing DDL Implicit Commits

LEVEL: ADVANCED

Demonstrate what happens when an engineer accidentally runs `CREATE TEMPORARY TABLE` or `ALTER TABLE` inside an open transaction. Explain how to rewrite it safely.

> **Requirements:** Identification and refactoring of implicit commit hazards.

13. Peer-to-Peer Wallet Transfer

LEVEL: ADVANCED

Simulate a digital wallet transfer between User A and User B with sender lock: `SELECT ... FOR UPDATE`. Include rollback if sender has insufficient funds.

> **Requirements:** Combine pessimistic locking with TCL rollback.

14. Ride-Sharing Fare Split

LEVEL: ADVANCED

Split a $60 taxi ride equally among 3 riders. Deduct $20 from each rider's card. If rider 3 card fails, abort the split entirely and restore riders 1 and 2 balances.

> **Requirements:** Multi-party atomicity guarantee.

15. Batch CSV Import with Partial Commit Strategy

LEVEL: ADVANCED

Import batches of 500 customers. Use savepoints per batch so that a corrupt row in batch 3 only discards batch 3 while allowing batches 1 and 2 to persist.

> **Requirements:** Chunked savepoint isolation pattern.

16. Event Ticket Overbooking Prevention

LEVEL: ADVANCED

Only 1 ticket remains for a concert. Two users submit requests simultaneously. Design a transaction with `WHERE tickets_available > 0` that safely commits for one and rolls back for the other.

> **Requirements:** Concurrency check with rollback.

17. Flight + Hotel Package Deal Booking

LEVEL: ADVANCED

A vacation package requires both a flight seat AND a hotel room. If hotel reservation fails after flight seat is booked, ensure full rollback so customer is never charged for half a vacation.

> **Requirements:** All-or-nothing package transaction design.

18. Nested Savepoint Overwrite Rules

LEVEL: ADVANCED

Create `SAVEPOINT check1;`, perform an update, then create another `SAVEPOINT check1;` with the same name. Explain what happens to the first savepoint and verify behavior.

> **Requirements:** Test and demonstrate duplicate savepoint name semantics in MySQL.

19. E-Commerce Order Cancellation & Restock

LEVEL: ADVANCED

When a customer cancels an order: mark order as 'CANCELLED', restore inventory counts, and issue wallet refund credit. Encapsulate in a single safe transaction.

> **Requirements:** Reversal transaction pattern with audit logging.

20. Distributed Microservice Saga Pattern in SQL

LEVEL: ADVANCED

Design a compensating transaction table in SQL to log executed steps and automated rollback actions in case downstream services crash.

> **Requirements:** Compensating transaction design pattern.